Catálogo gastronómico Express GO conectado a Supabase

Rodrigo Ledezma Sanchez

Universidad Privada Domingo Savio

Aplicaciones Móviles I

Docente Jimmy Nataniel Requena Llorentty

Santa Cruz de la Sierra Bolivia

13 de noviembre de 2026

Índice

- Resumen
- Introducción
- Objetivos y alcance
- Método de desarrollo
- Diseño de la base de datos
- Seguridad y políticas RLS
- Implementación de la aplicación Flutter
- Formularios y validaciones
- Pruebas y resultados
- Emprendedurismo responsable y ODS 8
- Limitaciones y trabajo futuro
- Conclusiones
- Referencias
- Anexos

Repositorio de la actividad

GitHub Aplicaciones móviles 1 Actividad 02

Resumen

Este informe documenta Express GO, una aplicación Flutter orientada a pequeños negocios gastronómicos que atienden entre las 00:00 y las 06:00 en Santa Cruz de la Sierra. La solución consulta seis locales y doce productos desde Supabase, muestra estados de carga, error y vacío, y permite registrar consultas y reservas mediante formularios validados. La conexión utiliza una clave publicable inyectada con dart define, mientras que las políticas de seguridad a nivel de fila restringen la lectura y la escritura según el tipo de operación. Las pruebas verificaron lectura pública, inserciones válidas, bloqueo de modificaciones no autorizadas, navegación, validación y comportamiento sin conexión. También se corrigió una carga indefinida observada en un dispositivo: todas las operaciones remotas tienen un límite de 15 segundos y la interfaz permite reintentar. La APK de versión final se instaló en un teléfono Android y abrió el catálogo real de Express GO.

Palabras clave: Flutter, Supabase, catálogo gastronómico, Row Level Security, validación, Express GO.

Introducción

Los pequeños negocios gastronómicos de Santa Cruz de la Sierra que atienden de madrugada suelen publicar sus productos en redes sociales y recibir pedidos o consultas por mensajería. Para el cliente resulta difícil saber qué local continúa abierto entre las 00:00 y las 06:00, qué productos ofrece y cómo comunicarse. Express GO adapta el caso académico ConectaCruz a este problema concreto: reúne un número reducido de locales nocturnos, productos, consultas y reservas en una experiencia móvil uniforme. En la Actividad 02 los datos dejan de estar escritos dentro de la aplicación y pasan a una base real en Supabase.

Supabase ofrece una API generada sobre PostgreSQL y un modelo de seguridad basado en Row Level Security. En una aplicación móvil la clave publicable puede formar parte del cliente, pero su alcance debe quedar limitado por políticas de base de datos; las claves secretas nunca deben distribuirse en la APK (Supabase, 2026a, 2026b). Esta diferencia es central para el proyecto: el cliente puede leer el catálogo e insertar una solicitud válida, pero no puede cambiar negocios ni consultar solicitudes de otras personas.

El trabajo se desarrolló con Flutter para Android, Supabase como backend, scripts SQL versionados y un repositorio en GitHub. Se aplicó una arquitectura sencilla por capas para separar modelos, pantallas y acceso a datos. Además, se documentaron pruebas automáticas y manuales para demostrar los estados solicitados por la rúbrica.

Objetivos y alcance

Objetivo general

Implementar el catálogo móvil de Express GO conectado a Supabase para leer locales gastronómicos y productos reales, registrar consultas y reservas válidas, y proteger los datos mediante políticas RLS.

Objetivos específicos

- Crear y poblar las tablas necesarias mediante scripts SQL reproducibles.
- Mostrar carga, error, vacío y datos en el catálogo usando FutureBuilder.
- Navegar desde la bienvenida al catálogo y desde cada negocio a su detalle.
- Validar nombre, teléfono, consulta, cantidad de personas, fecha y hora.
- Evitar envíos duplicados deshabilitando el botón mientras se realiza cada inserción.

- Evitar esperas indefinidas mediante límites de tiempo y recuperación manual.
- Comprobar que el cliente público no puede modificar información ajena.

Alcance de esta entrega

La versión incluye lectura del catálogo, búsqueda local, detalle de productos, envío de consultas y reserva con fecha futura. Los pedidos, la autenticación de clientes, la gestión de propietarios y las notificaciones se dejan fuera del alcance para mantener una versión mínima verificable.

Método de desarrollo

El trabajo se organizó en cuatro etapas. Primero se revisó la consigna y el material del taller para identificar tablas, políticas, validaciones y evidencias. Segundo se crearon los scripts 01 a 03 y se ejecutaron en el proyecto de producción. Tercero se reemplazó la lista local por un repositorio de datos que usa supabase flutter. Finalmente se realizaron análisis estático, pruebas de widgets, pruebas de validadores y pruebas manuales en un emulador Android.

Etapa	Actividad	Evidencia
1	Definición del modelo y reglas	Seis tablas y validaciones
2	Ejecución de scripts SQL	6 negocios y 12 productos
3	Integración Flutter	Catálogo, detalle y formularios
4	Verificación	Pruebas, capturas y control RLS

Arquitectura aplicada

La interfaz llama a una abstracción denominada ConectaCruzRepository, nombre interno heredado del ejercicio académico. SupabaseRepository implementa esa interfaz y concentra las consultas e inserciones de Express GO. De este modo, las pantallas no dependen directamente

del cliente de Supabase y pueden probarse con un repositorio falso. Los modelos Negocio y Producto convierten las respuestas JSON en objetos de Dart.

Capa	Responsabilidad	Archivos principales
Presentación	Pantallas, navegación y estados	lib/pages y lib/widgets
Dominio	Entidades y validadores	lib/models y lib/core
Datos	Contrato y acceso a Supabase	lib/services
Configuración	Variables dart define	lib/config/config.dart
Persistencia	Tablas, semillas y RLS	supabase/*.sql

Diseño de la base de datos

El esquema contempla el catálogo actual y una ampliación futura hacia pedidos. Todas las tablas incluyen claves primarias y relaciones. Las restricciones de formato también se aplican en PostgreSQL para que la seguridad no dependa únicamente de la interfaz móvil.

Tabla	Propósito	Acceso público
negocios	Datos de cada negocio y estado activo	Lectura limitada
productos	Productos, precios y disponibilidad	Lectura limitada
consultas	Preguntas enviadas por clientes	Solo inserción válida
reservas	Fecha, hora y cantidad de personas	Solo inserción válida
pedidos	Cabecera de pedidos futuros	Sin acceso público
pedido items	Detalle de productos por pedido	Sin acceso público

Scripts ejecutados

01 tablas: crea las seis tablas, relaciones, restricciones y marcas de tiempo.

02 RLS lectura: activa RLS y habilita lectura de negocios activos y productos disponibles.

02b datos semilla: inserta seis locales gastronómicos nocturnos y doce productos de Express GO.

03 escritura pública: permite únicamente consultas y reservas que cumplen las reglas definidas.

Seguridad y políticas RLS

RLS añade una condición de autorización a cada operación. Activar RLS no basta: también se revocaron privilegios generales y después se concedieron operaciones específicas a los roles anon y authenticated. La política de lectura de negocios exige que el registro esté activo; la de productos exige disponibilidad y que el negocio relacionado permanezca activo.

Las tablas consultas y reservas no tienen una política pública de lectura. El cliente solo recibe permiso de inserción y la política comprueba que el nombre tenga longitud válida, que el teléfono tenga ocho dígitos y empiece con 6 o 7, y que el negocio exista y esté activo. En reservas también se verifica una cantidad de una a cincuenta personas y una fecha y hora posteriores al momento actual de Santa Cruz.

Prueba con clave pública	Resultado	Interpretación
GET negocios	6 filas	Lectura permitida
GET productos	12 filas	Lectura permitida
POST consulta válida	HTTP 201	Inserción permitida
POST reserva futura	HTTP 201	Inserción permitida
PATCH negocio	HTTP 401	Modificación bloqueada
GET consultas y reservas	Sin política	Datos privados

La demostración de bloqueo es importante porque una clave publicable no es un secreto. Cualquier persona podría extraerla de una APK; por ello, la seguridad real se encuentra en los privilegios y políticas de PostgreSQL, no en ocultar la clave del cliente. La aplicación recibe la URL y la clave mediante SUPABASE URL y SUPABASE PUBLISHABLE KEY en dart define.

Implementación de la aplicación Flutter

Catálogo remoto y FutureBuilder

CatalogoPage es un StatefulWidget. En initState se crea una sola vez el Future que solicita los negocios. FutureBuilder observa ese Future y construye una interfaz distinta para carga, error y resultado. Esta decisión evita repetir la consulta cada vez que Flutter reconstruye el árbol por una búsqueda o un cambio visual. El botón Actualizar y el gesto de recarga crean un nuevo Future solo cuando el usuario lo solicita.

Cuando la consulta termina, la búsqueda se aplica localmente sobre nombre, rubro y descripción. Si Supabase devuelve una lista vacía se informa que aún no existen negocios. Si la búsqueda no coincide se muestra Sin resultados. Si no hay conexión se presenta un mensaje específico y un botón Reintentar.

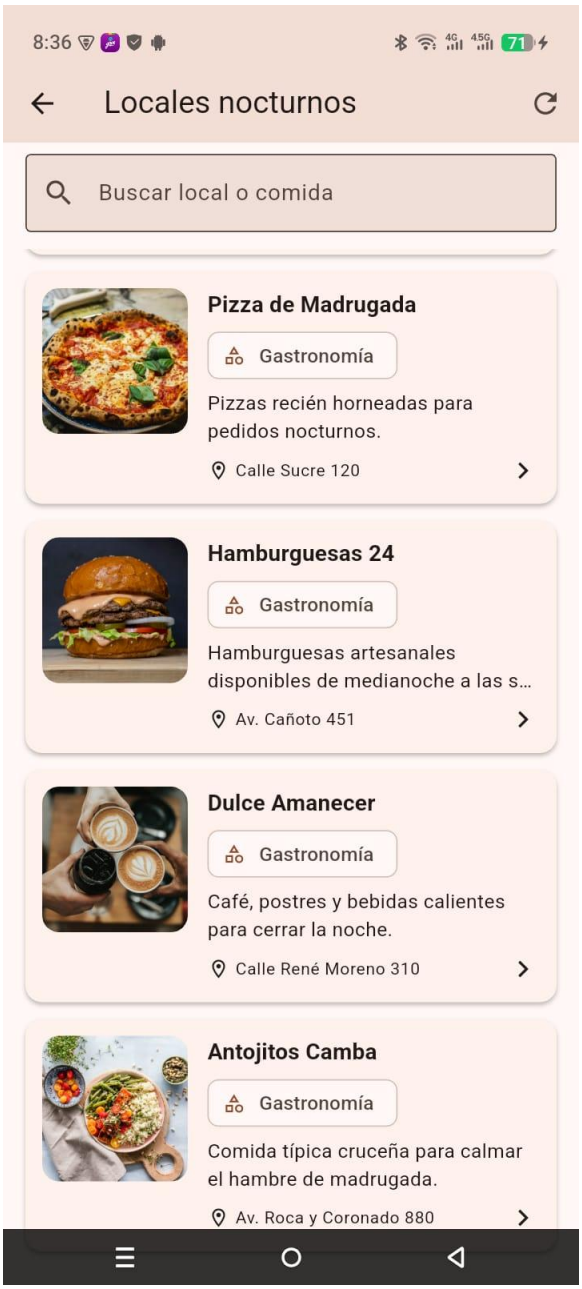


Figura 1. *Catálogo de Express GO cargado desde Supabase en un dispositivo Android.*

Corrección de la carga indefinida

Durante la verificación en un dispositivo se observó que el catálogo podía permanecer en el indicador de carga. La causa técnica era que las operaciones remotas esperaban la respuesta de Supabase sin un límite temporal. En una red inestable, una conexión iniciada pero no completada podía dejar el Future pendiente y FutureBuilder continuaba mostrando el estado de espera.

La corrección se aplicó en SupabaseRepository. Las lecturas de negocios y productos, y las inserciones de consultas y reservas, usan un timeout de 15 segundos. Si la red no responde, el Future termina con una excepción controlada y CatalogoPage cambia a un estado de error con explicación y botón Reintentar. La consulta sigue creándose una sola vez en initState; únicamente se reemplaza cuando el usuario solicita actualizar o reintentar.

Después de la modificación se generó una APK release con la URL y la clave publicable recibidas mediante dart define. La APK se instaló primero en un emulador limpio y después en un teléfono Android. En ambos casos se abrió la pantalla de bienvenida y se ingresó al catálogo. La Figura 2 confirma la identidad visual de Express GO en el dispositivo, mientras que la Figura 1 demuestra la carga de los locales reales; la aplicación ya no quedó detenida en la carga durante la prueba final.

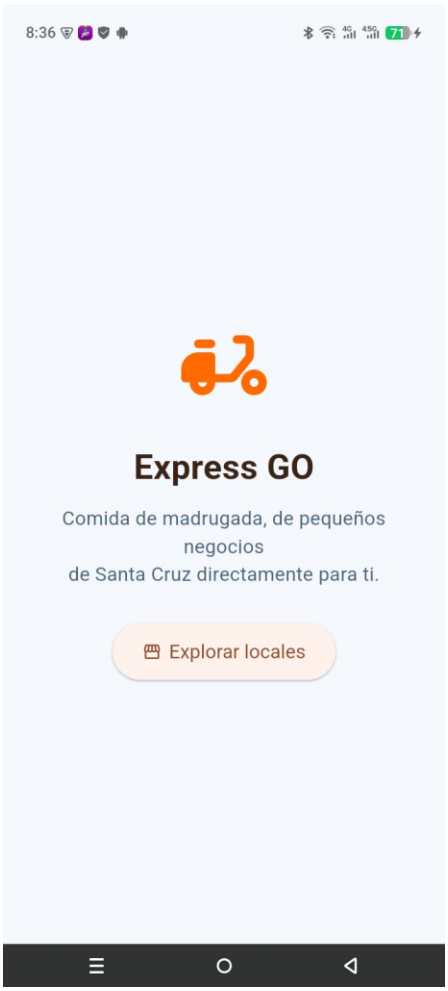


Figura 2. *Pantalla de bienvenida de la APK release instalada en un dispositivo Android.*

Detalle y navegación

Cada tarjeta abre DetalleNegocioPage y pasa el negocio seleccionado. La pantalla muestra imagen, rubro, descripción, dirección y teléfono. Después solicita únicamente los productos relacionados con el identificador del negocio. Ese segundo Future también se crea una sola vez en initState y contempla carga, error y ausencia de productos.

Los botones Consultar y Reservar abren formularios separados. La navegación mantiene el flujo simple y permite regresar sin perder el catálogo previamente cargado. Esta organización también facilita agregar una pantalla de pedidos más adelante.

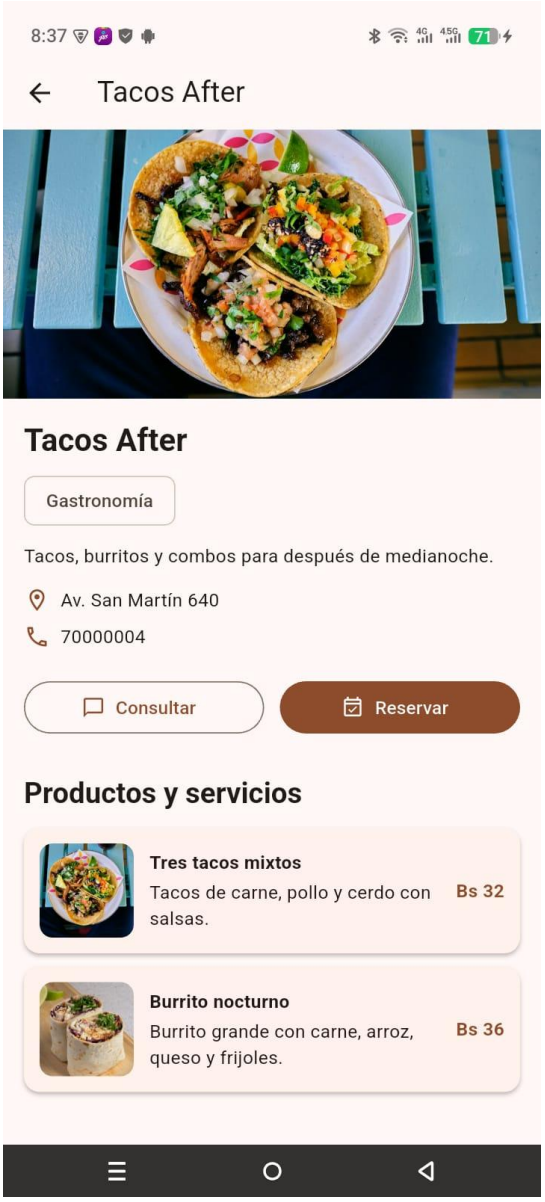


Figura 3. *Detalle de Tacos After con productos y precios obtenidos de Supabase.*

Formularios y validaciones

Los formularios usan Form y GlobalKey para ejecutar validaciones antes de enviar. Cada campo muestra una etiqueta persistente, teclado apropiado y mensaje de error concreto. Durante la petición, la variable enviando desactiva el botón y reemplaza el ícono por un indicador de progreso. Este comportamiento evita toques repetidos y comunica que la operación continúa.

Campo	Regla	Mensaje esperado
Nombre	Mínimo 2 caracteres	Ingresa tu nombre
Teléfono	8 dígitos; inicia en 6 o 7	Usa 8 dígitos
Consulta	Mínimo 10 caracteres	Escribe una consulta más completa
Personas	Entre 1 y 50	Cantidad válida
Fecha y hora	Debe ser futura	Selecciona un horario futuro

La validación se repite en el servidor mediante las políticas RLS. Este enfoque de doble capa mejora la experiencia porque detecta errores antes del envío y, al mismo tiempo, impide que un cliente modificado omita las reglas.



Figura 4. *Formulario de consulta de Express GO ejecutándose en el dispositivo.*

La consulta se completó y se envió desde la APK release. Después de recibir HTTP 201, la pantalla regresó al detalle de Tacos After y mostró la confirmación de la Figura 5.

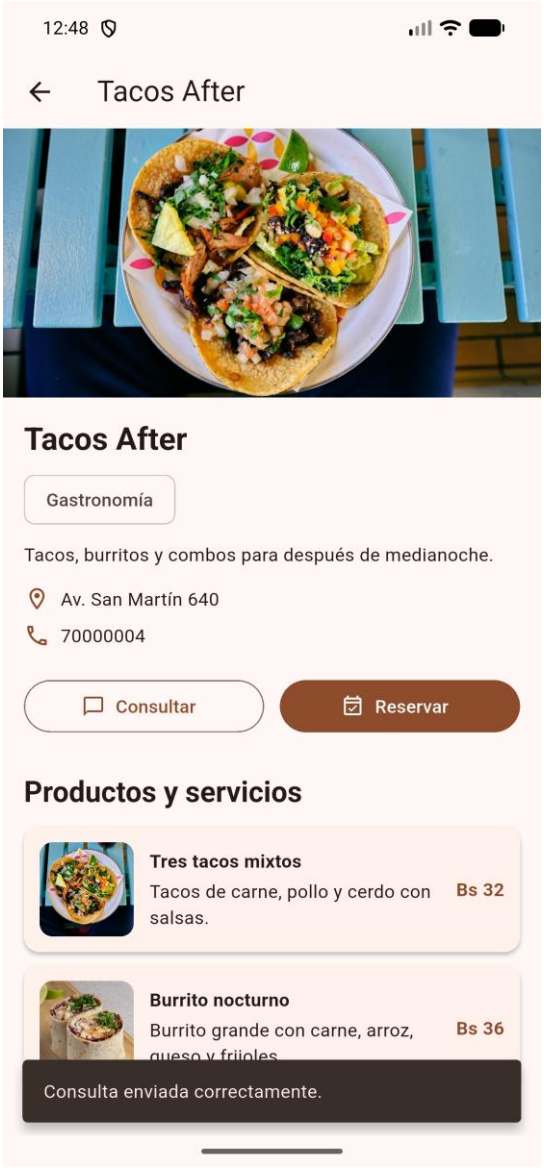


Figura 5. *Confirmación de una consulta guardada en Supabase desde Express GO.*

Reserva con fecha y hora

El formulario de reserva usa selectores nativos para fecha y hora. La fecha mínima es el día actual y el valor combinado se verifica nuevamente antes de enviarse. La cantidad de personas se convierte a número entero solo después de validar el campo. Finalmente el repositorio transforma DateTime a los tipos date y time esperados por PostgreSQL.



Figura 6. *Formulario de reserva con nombre, teléfono, personas, fecha y hora.*

Además de mensajes claros, se añadieron hints de autocompletado, etiquetas semánticas en tarjetas, tooltips en acciones y tamaños de toque amplios. Estas decisiones apoyan accesibilidad básica y reducen errores de entrada.

La reserva se realizó para Tacos After con fecha y hora futuras. La Figura 7 confirma el resultado visible después de que Supabase aceptó la inserción.

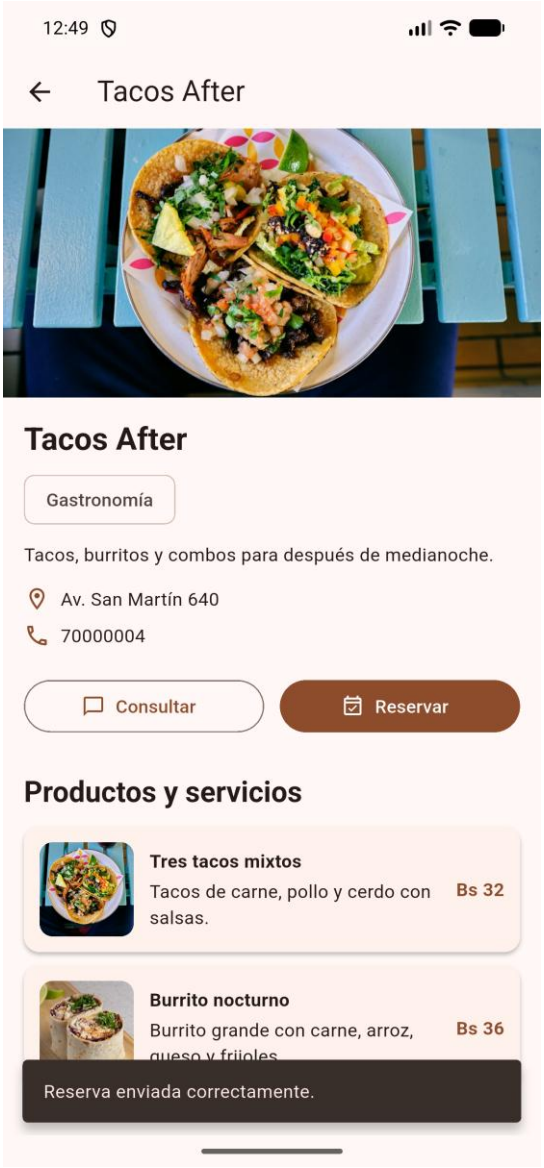


Figura 7. *Confirmación de una reserva guardada en Supabase desde Express GO.*

Pruebas y resultados

Pruebas automáticas

El análisis estático se ejecutó sin problemas. Las seis pruebas automatizadas cubren validadores de teléfono, mensaje y personas; navegación desde la bienvenida; carga única del catálogo y de productos; y errores del formulario de consulta. Se utilizó un repositorio falso para probar la interfaz sin depender de internet.

Comando	Resultado
flutter analyze no pub	Sin problemas
flutter test no pub	6 pruebas aprobadas
flutter build apk release	APK generada, instalada y conectada

Pruebas manuales

Escenario	Resultado observado
Carga inicial	Indicador de progreso
Datos reales	6 negocios y 12 productos
Búsqueda sin coincidencia	Estado Sin resultados
Sin internet	Mensaje y botón Reintentar
Respuesta remota demorada	Timeout a los 15 segundos
APK release	Catálogo real con 6 negocios
Consulta válida	Inserción y confirmación
Reserva futura	Inserción en tabla reservas
Edición no autorizada	HTTP 401

Resultados en Supabase

La consulta administrativa al proyecto de producción confirmó seis negocios, doce productos, tres consultas y tres reservas. Una inserción de cada tipo se realizó por API para verificar la política y dos se realizaron desde la aplicación para comprobar el flujo completo. Las inserciones más recientes corresponden a Tacos After y se ejecutaron desde la APK release de Express GO.

Tabla	Cantidad final	Origen
negocios	6	Datos semilla
productos	12	Datos semilla

Tabla	Cantidad final	Origen
consultas	3	API y aplicación
reservas	3	API y aplicación

La Figura 8 muestra el panel oficial del proyecto Supabase de producción. El listado confirma seis tablas y los conteos registrados: tres consultas, seis negocios, doce productos y tres reservas. Los estados de interfaz también quedaron documentados: la Figura 9 demuestra el estado vacío de una búsqueda y la Figura 10 reproduce la falta de conectividad del emulador con la acción de recuperación.

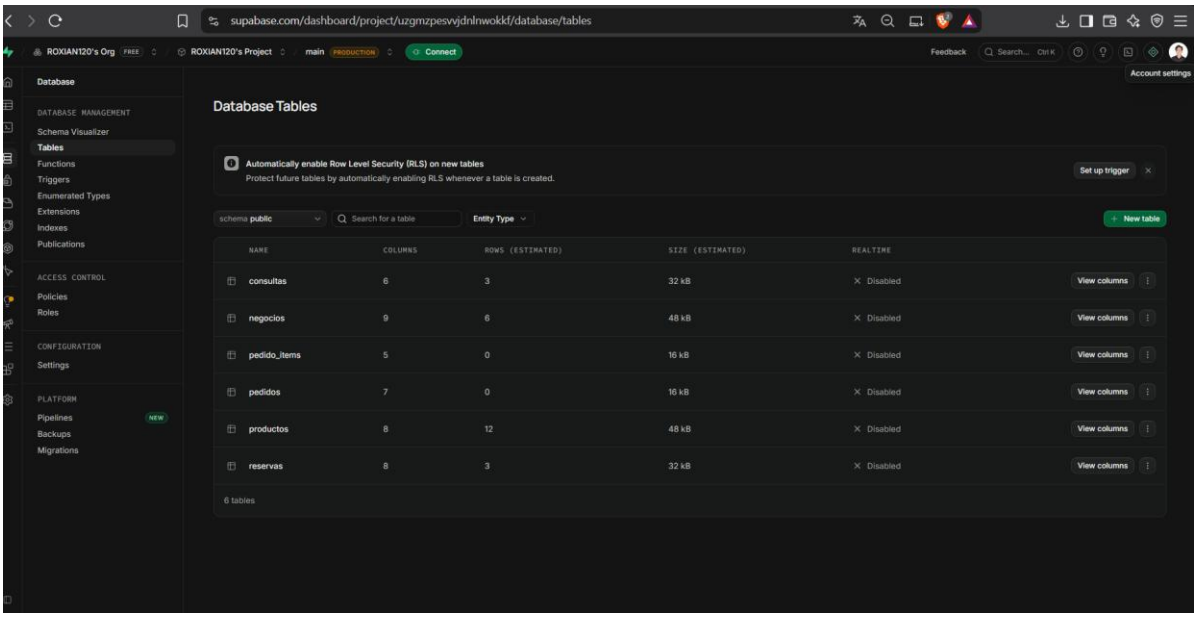


Figura 8. *Panel oficial de Supabase con las tablas y sus conteos verificados.*

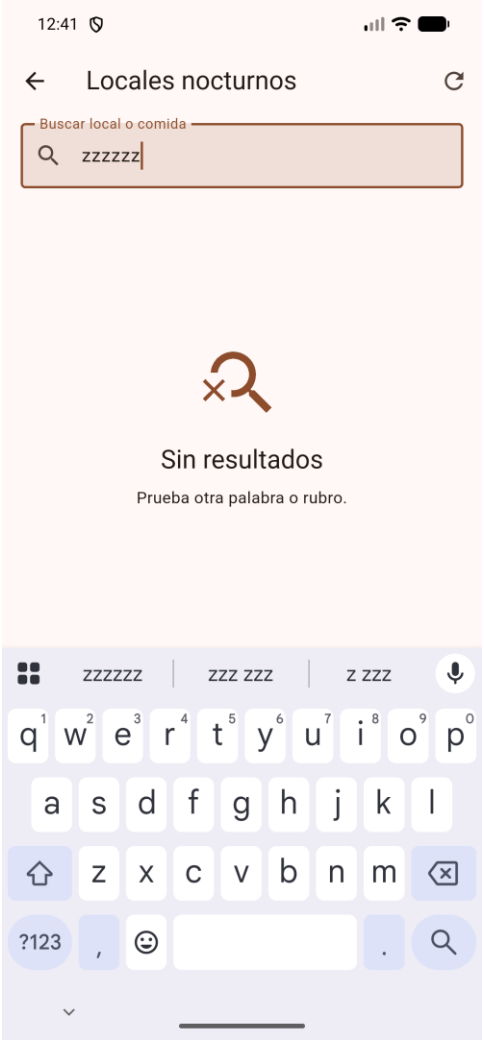


Figura 9. *Estado vacío cuando la búsqueda no coincide con ningún local.*

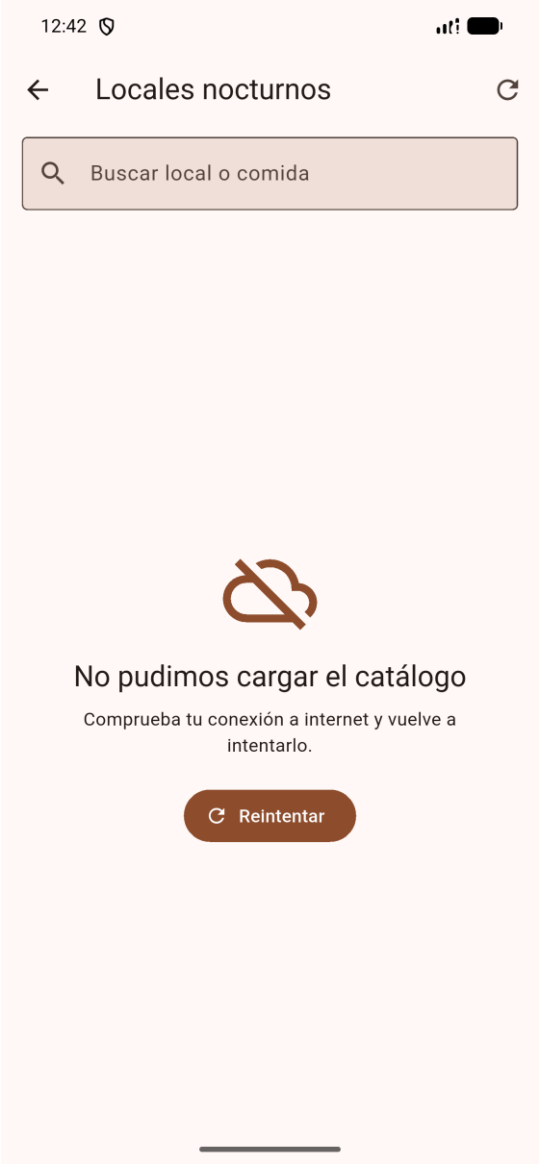


Figura 10. *Estado de error con mensaje comprensible y botón de reintento.*

Emprendedurismo responsable y ODS 8

Express GO se relaciona con el ODS 8 porque facilita que pequeños emprendimientos gastronómicos nocturnos muestren su oferta y reciban oportunidades comerciales mediante herramientas digitales. Un catálogo común reduce la barrera de crear una aplicación individual para cada local y puede mejorar la visibilidad de negocios familiares que atienden en una franja horaria con poca oferta organizada.

El enfoque responsable exige proteger la información de clientes. Por ello, las consultas y reservas no son públicas, la aplicación no incluye claves secretas y la base limita las operaciones del cliente. También se evita recolectar datos innecesarios: para esta versión solo se solicita nombre, teléfono y la información indispensable para responder o reservar.

La adaptabilidad digital aparece en la separación entre interfaz y repositorio. La misma aplicación puede ampliar productos, autenticación o paneles administrativos sin reescribir todas las pantallas. La documentación de scripts y pruebas facilita que otro integrante reproduzca el entorno y comprenda las decisiones tomadas.

Limitaciones y trabajo futuro

La versión final se verificó en un emulador Android y la incidencia reportada en un dispositivo motivó la incorporación del timeout. Sin embargo, todavía conviene repetir una matriz de pruebas en varios teléfonos reales, versiones de Android, tamaños de pantalla y calidades de red. Las imágenes de los datos semilla dependen de servicios externos, por lo que en producción convendría usar Supabase Storage y miniaturas optimizadas.

La inserción pública controlada es apropiada para una demostración, pero una versión comercial debería incorporar autenticación, límites de frecuencia y mecanismos contra spam. También se requiere un panel seguro para que los negocios respondan consultas, confirmen reservas y actualicen su catálogo.

- Agregar autenticación de clientes y propietarios.
- Crear políticas RLS por propietario y pruebas con más de un usuario.
- Implementar pedidos, estados y notificaciones.
- Añadir caché local y una estrategia offline más completa.
- Realizar pruebas de accesibilidad y rendimiento en dispositivos físicos.

Conclusiones

La Actividad 02 convirtió Express GO en una aplicación conectada a datos reales. Flutter obtiene seis locales gastronómicos nocturnos y doce productos mediante un repositorio Supabase, presenta estados completos de interfaz y mantiene la navegación entre bienvenida, catálogo y detalle. Los formularios validan información, bloquean el botón durante el envío y guardan consultas y reservas en sus tablas.

La seguridad se implementó con RLS y privilegios mínimos. La prueba con la clave publicable confirmó lecturas autorizadas e inserciones válidas, mientras que el intento de modificar un negocio fue rechazado con HTTP 401. El resultado satisface los criterios principales de la rúbrica y deja evidencia reproducible en SQL, código, pruebas y capturas.

La corrección del límite de espera completó el comportamiento de recuperación solicitado por la rúbrica. La aplicación no mantiene indefinidamente el indicador de carga: ante una respuesta demorada pasa a error, explica la situación y permite reintentar. La instalación de la APK release y la carga efectiva de los seis negocios demuestran que el entregable ejecutable utiliza los datos reales del proyecto.

Referencias

Flutter. (2026). FutureBuilder class. https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html

Naciones Unidas. (s. f.). Objetivo 8 Trabajo decente y crecimiento económico. https://sdgs.un.org/goals/goal8

Requena Llorentty, J. N. (2026). Taller Actividad 02 ConectaCruz Supabase [Diapositivas]. Universidad Privada Domingo Savio.

Supabase. (2026a). Flutter quickstart. https://supabase.com/docs/guides/getting-started/quickstarts/flutter

Supabase. (2026b). Understanding API keys. https://supabase.com/docs/guides/getting-started/api-keys

Supabase. (2026c). Row level security. https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase. (2026d). Tables and data. https://supabase.com/docs/guides/database/tables

Anexos

Anexo A Estructura de evidencias

El repositorio incluye capturas numeradas de bienvenida, catálogo, detalle, formularios, envíos, estado vacío, estado sin conexión, confirmaciones de escritura y el panel oficial de Supabase con los conteos de las tablas. También contiene resultado pruebas.md con el resumen de filas, la prueba RLS y la corrección de carga indefinida.

Anexo B Enlace del repositorio

https://github.com/ROXIAN120/Aplicaciones-moviles-1#actividad-02--express-go-con-supabase